Git Introduction

What is Git?

Git is a **distributed version control system** that helps manage and track changes to files in a project. It is widely used in software development to maintain the history of changes, collaborate with others, and revert to previous versions when necessary.

Key Concepts in Git

- **Repository (Repo)**: A directory where your project files and their revision history are stored.
- **Commit**: A snapshot of changes made to the files. Each commit has a unique ID.
- **Branch**: A separate line of development. The main branch is called `master` or `main`.
- **Merge**: Integrating changes from one branch into another.
- **Clone**: Creating a copy of a repository.
- **Push**: Uploading local commits to a remote repository.
- **Pull**: Fetching and merging changes from a remote repository to your local copy.

Git Installation

Installing Git

PATHNEX
Shaping Careers, Building Futures

1. **Windows**: Download from git-scm.com. The installer will guide you through the installation.
2. **Mac**: You can install Git via Homebrew or download from the Git website.
3. **Linux**: Use your package manager, e.g., `sudo apt-get install git` on Ubuntu.

Basic Git Commands

1. git config

Set up your name and email for Git. This will be used for your commits.

```
bash
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

2. git init

Initialize a new Git repository in the current directory.

```
bash
git init
```

3. git clone

Clone an existing remote repository to your local machine.

```
bash
git clone <repository_url>
```

4. git status

Check the status of your working directory, showing tracked and untracked files.

```
bash
git status
```

5. git add

Stage changes for commit.

```
bash
git add <filename>      # To add a single file
git add .               # To add all files
```

6. git commit

Commit changes with a message.

```
bash
git commit -m "Your commit message"
```

7. git log

View the commit history.

```
bash
git log
```

8. git diff

See the difference between changes made in the working directory and the latest commit.

```
bash
git diff
```

Working with Branches

1. git branch

List all branches or create a new branch.

```
bash
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

```
git branch            # List branches
git branch <branch_name>    # Create a new branch
```

2. git checkout

Switch to another branch.

```
bash
git checkout <branch_name>
```

3. git merge

Merge changes from another branch into the current branch.

```
bash
git merge <branch_name>
```

Working with Remote Repositories

1. git remote add

Add a remote repository URL.

```
bash
git remote add origin <repository_url>
```

PATHNEX
Shaping Careers, Building Futures

2. git push

Push commits to a remote repository.

```
bash
git push origin <branch_name>
```

3. git pull

Fetch changes from a remote repository and merge them into your local branch.

```
bash
git pull origin <branch_name>
```

Common Workflow in Git

1. **Start a project**: Initialize the repository or clone an existing one.

   ```
   bash
   git init    # or git clone <repository_url>
   ```

2. **Make changes**: Modify files or add new ones.
3. **Track changes**: Stage files and commit them.

```
bash
git add <filename>
git commit -m "message"
```

4. **Collaborate**: Push and pull changes to/from the remote repository.

```
bash
git push origin main
git pull origin main
```

5. **Branching and merging**: Create branches to work on features and merge them back into the main branch.

```
bash
git branch <feature_branch>
git checkout <feature_branch>
# Work on feature...
git commit -m "Feature completed"
git checkout main
git merge <feature_branch>
```

Git Best Practices

- **Commit often, but with meaning**: Make small, descriptive commits.
- **Write clear commit messages**: Describe what and why, not just "fixed bug".
- **Use branches**: Create separate branches for each feature or bug fix.
- **Pull often**: Regularly pull changes from the remote repository to avoid conflicts.
- **Push early and often**: Keep your remote repository up to date with your local changes.

PATHNEX
Shaping Careers, Building Futures

🛠️ Steps to Configure Git (First-Time Setup)

1. Install Git

- **Windows:** git-scm.com → Download & install.
- **macOS:** Use Homebrew: `brew install git`
- **Linux:** Use package manager:
  - Debian/Ubuntu: `sudo apt install git`
  - Fedora: `sudo dnf install git`

---

2. Verify Installation

```
git --version
```

---

3. Set Global Username and Email

These are used in your commits.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

---

4. Check Current Configuration

```
git config --list
```

---

5. (Optional) Set Default Text Editor

```
git config --global core.editor "code"      # For VS Code
```

---

6. (Optional) Enable Colored Output

```
git config --global color.ui auto
```

---

7. (Optional) Set Default Branch Name

Git 2.28+ allows you to set the default branch name:

```
git config --global init.defaultBranch main
```

---

8. (Optional) Configure SSH (for GitHub/GitLab)

1. Generate SSH key:
2. `ssh-keygen -t ed25519 -C "your.email@example.com"`
3. Add the key to your GitHub/GitLab account.
4. Test the connection:

   ```
   ssh -T git@github.com
   ```

PATHNEX
Shaping Careers, Building Futures